Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka Techniczna

Implementacja i analiza efektywności algorytmu podziału i ograniczeń

Imię i nazwisko: Wiktor Zięba
Numer albumu: 281056

Wrocław, kwiecień 2026

Spis treści

1 Wstęp teoretyczny **2**

2 Opis testowanego algorytmu **2**
2.1 Idea metody Branch and Bound . . . 2
2.2 Dolne ograniczenie kosztu . . . 2
2.3 Ograniczenie górne i rozwiązanie początkowe . . . 3
2.4 Strategia przeszukiwania Breadth First Search . . . 3
2.5 Strategia przeszukiwania Best First Search . . . 3

3 Przykład praktyczny **4**

4 Plan eksperymentu **7**
4.1 Cel eksperymentu . . . 7
4.2 Zakres badanych rozmiarów . . . 8
4.3 Liczba powtórzeń i limit czasu . . . 8
4.4 Wykonane pomiary . . . 8
4.5 Organizacja pomiarów . . . 9

5 Wyniki eksperymentów **9**
5.1 Porównanie czasów działania badanych wariantów . . . 9
5.2 Wpływ rozwiązania początkowego . . . 11
5.3 Strategia B&B BestFirstSearch dla większych wartości N . . . 13
5.4 Podsumowanie . . . 14

6 Literatura **15**

1 Wstęp teoretyczny

Problem komiwojażera to zagadnienie optymalizacyjne polegające na znalezieniu w pełnym grafie ważonym najkrótszej ścieżki począwszy od punktu startowego grafu która przechodzi przez wszystkie inne wierzchołki dokładnie raz i kończy się w punkcie startowym (cykl Hamiltona). W przypadku asymetrycznego problemu komiwojażera, który jest przedmiotem badania koszt przejścia z wierzchołka X do wierzchołka Y może mieć różną wartość od kosztu przejścia z wierzchołka Y do X. Istnieją dwa podstawowe podejścia rozwiązywania problemu komiwojażera:

- **Algorytmy dokładne** – wyznaczają rozwiązanie optymalne, jednak ich złożoność czasowa rośnie bardzo szybko wraz ze wzrostem liczby wierzchołków co sprawia, że dla większych instacji problemu ich zastosowanie staje się niepraktyczne
- **Algorytmy heurystyczne** - algorytmy pozwalające uzyskać wynik w znacznie krótszym czasie kosztem tego, że nie gwarantują znalezienia rozwiązania optymalnego

Problem komiwojażera należy do klasy problemów NP-trudnych, nie jest znany algorytm wielomianowy znajdujący optymalne rozwiązanie dla każdego przypadku. Wraz ze wzrostem ilości miast, liczba możliwych tras rośnie silniowo dla metod opartych na pełnym przeglądzie, co czyni obliczenie optymalnej trasy niemożliwym dla dużych zbiorów danych. Jednym ze sposobów ograniczenia liczby analizowanych rozwiązań jest zastosowanie metody podziału i ograniczeń. Metoda ta polega na przeszukiwaniu drzewa rozwiązań częściowych oraz odrzucaniu tych gałęzi, które na podstawie oszacowanego dolnego ograniczenia kosztu nie mogą prowadzić do rozwiązania lepszego od już znalezionego. Dzięki temu możliwe jest uzyskanie rozwiązania optymalnego przy mniejszej liczbie analizowanych przypadków niż w przeglądzie zupełnym.

2 Opis testowanego algorytmu

W ramach projektu zaimplementowano algorytm dokładny Branch and Bound przeznaczony do rozwiązywania asymetrycznego problemu komiwojażera. Zastosowana metoda nie polega na pełnym przeglądzie wszystkich możliwych permutacji odwiedzania miast, zamiast tego przeszukuje drzewo rozwiązań częściowych oraz odrzuca gałęzie, które nie mają potencjału na wynik lepszy od już znalezionego. W odróżnieniu od algorytmu Brute Force, który sprawdza wszystkie możliwe trasy, Branch and Bound wykorzystuje mechanizm ograniczeń pozwalający zmniejszyć liczbę analizowanych przypadków. Dla każdego rozważanego węzła drzewa wyznaczane jest dolne ograniczenie kosztu. Jeżeli wartość tego ograniczenia jest większa lub równa kosztowi najlepszego dotychczas znalezionego rozwiązania, dalsze rozwijanie danej gałęzi nie ma sensu i zostaje ona odrzucona. Pesymistyczna złożoność czasowa algorytmu Branch and Bound nadal wynosi O(n!) tak samo jak przegląd zupełnego, jednak w praktyce odcinanie gałęzi może znacząco skrócić czas działania. Efektywność algorytmu zależy przede wszystkim od jakości wyznaczanego ograniczenia dolnego, sposobu pozyskania rozwiązania początkowego oraz od przyjętej strategii wyboru kolejnych węzłów do rozwijania.

2.1 Idea metody Branch and Bound

W zaimplementowanym rozwiązaniu każdy węzeł drzewa to częściowo zbudowana trasa. Każdy węzeł przechowuje informacje o aktualnym wierzchołku, dotychczasowej ścieżce, aktualnym koszcie trasy, wartości dolnego ograniczenia kosztu, zredukowanej macierzy kosztów i zbiorze zawierającym informacje czy dane miasta zostały odwiedzone. Korzeń drzewa tworzony na początku odpowiada stanowi położenia w punkcie startowym, bez istniejącej trasy.

2.2 Dolne ograniczenie kosztu

Dolne ograniczenie (Lower Bound) to element algorytmu pozwalający oszacować minimalny możliwy koszt dowolnego pełnego rozwiązania rozwijanego z danego węzła. W zaimplementowanej wersji dolne ograniczenie jest wyznaczane na podstawie redukcji macierzy według następującego algorytmu:

1. Wyszukiwanie najmniejszej wartości w każdym wierszu
2. Odjęcie znalezionej w punkcie 1 wartości od wszystkich elementów wierszu
3. Analogiczne wyszukanie najmniejszej wartości w każdej kolumnie i odjęcie jej

W przypadku macierzy początkowej suma wszystkich odjętych wartości stanowi początkowe dolne ograniczenie kosztu. W kolejnych węzłach koszt redukcji jest jednym ze składników nowego ograniczenia dolnego.

2.3 Ograniczenie górne i rozwiązanie początkowe

Ograniczenie górne (Upper Bound) stosuje się w celu zwiększenia skuteczności odcinania gałęzi. Ograniczenie górne jest równe kosztowi najlepszego pełnego rozwiązania znalezionego do danego momentu. W projekcie zaimplementowano dwa możliwe warianty uruchomienia algorytmu: bez rozwiązania początkowego i z rozwiązaniem początkowym wyznaczonym heurystycznie, w którym przed rozpoczęciem przeszukiwania wyznaczana jest przykładowa trasa startowa za pomocą algorytmu Nearest Neigbour.

2.4 Strategia przeszukiwania Breadth First Search

W strategii Breadth First Search węzły drzewa rozwiązań rozwijane są poziomami. Najpierw analizowane są wszystkie częściowe rozwiązania zawierające jeden wybrany krok od wierzchołka startowego, następnie wszystkie rozwiązania zawierające dwa kroki, potem trzy itd. W praktyce algorytm przechodzi przez drzewo wszerz.

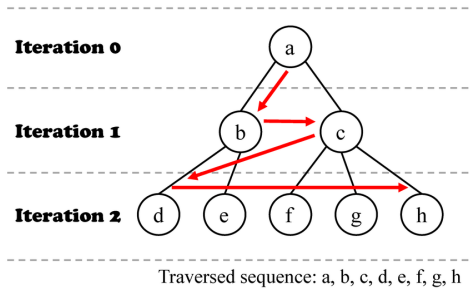

Figure 1: Przykład działania BFS

2.5 Strategia przeszukiwania Best First Search

Strategia w której kolejny rozwijany węzeł wybierany jest na podstawie najmniejszej wartości dolnego ograniczenia. W pierwszej kolejności analizowane są rozwiązania, które według aktualnego oszacowania mają największą szansę doprowadzić do rozwiązania optymalnego. W implementacji wykorzystano kolejkę priorytetową uporządkowaną według wartości ograniczenia dolnego. Dzięki temu algorytm koncentruje się na gałęziach mających największy potencjał.

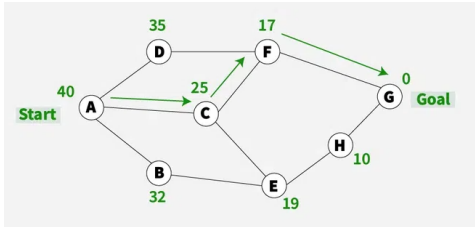

Figure 2: Przykład działania BestFS

3 Przykład praktyczny

W przykładzie użyto strategii Best First Search. Rozważana macierz ma postać:

$$\begin{bmatrix} -1 & 28 & 10 & 56 \\ 10 & -1 & 20 & 30 \\ 35 & 56 & -1 & 12 \\ 13 & 18 & 25 & -1 \end{bmatrix}$$

Krok 1. Wyznaczenie początkowego dolnego ograniczenia

W pierwszym kroku wykonano redukcję wierszy. Dla każdego wiersza znaleziono najmniejszą wartość, zwiększono ograniczenie dolne o tą wartość i odjęto ją od każdej liczby znajdującej się w tym wierszu. Następnie wykonano redukcję kolumn powtarzając analogiczny proces. W efekcie początkowe dolne ograniczenie dla korzenia drzewa rozwiązań wyniosło 50

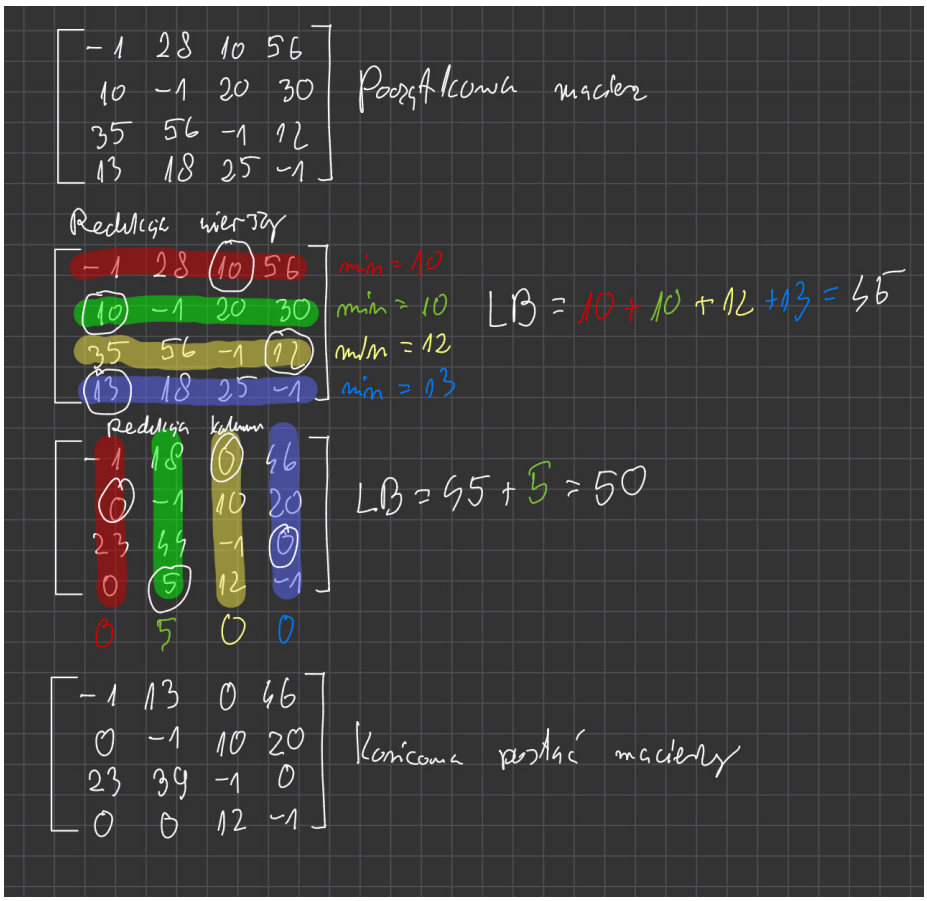

Figure 3: Wyznaczenie początkowego dolnego ograniczenia na podstawie redukcji macierzy kosztów

Krok 2. Rozwijanie drzewa rozwiązań według BestFS

Po wyznaczeniu ograniczenia dolnego dla korzenia wygenerowano pierwszy poziom drzewa. Dla każdego dziecka wartość ograniczenia dolnego obliczano jako sumę dwóch składników:

$$LB = \text{LB rodzica} + \text{wartość wybranego przejsścia w macierzy zredukowanej} + \text{dodatkowa redukcja}$$

Po wyborze kolejnego przejścia z macierzy usuwa się wiersz odpowiadający miastu, z którego wychodzimy, oraz kolumnę odpowiadającą miastu, do którego wchodzimy. Dodatkowo blokowany jest powrót do

poprzednich miast w sposób, który zapobiega tworzeniu przedwczesnych cykli. Dla tak zmodyfikowanej macierzy ponownie wykonuje się redukcję wierszy i kolumn, a suma odjętych wartości stanowi nowe ograniczenie dolne.

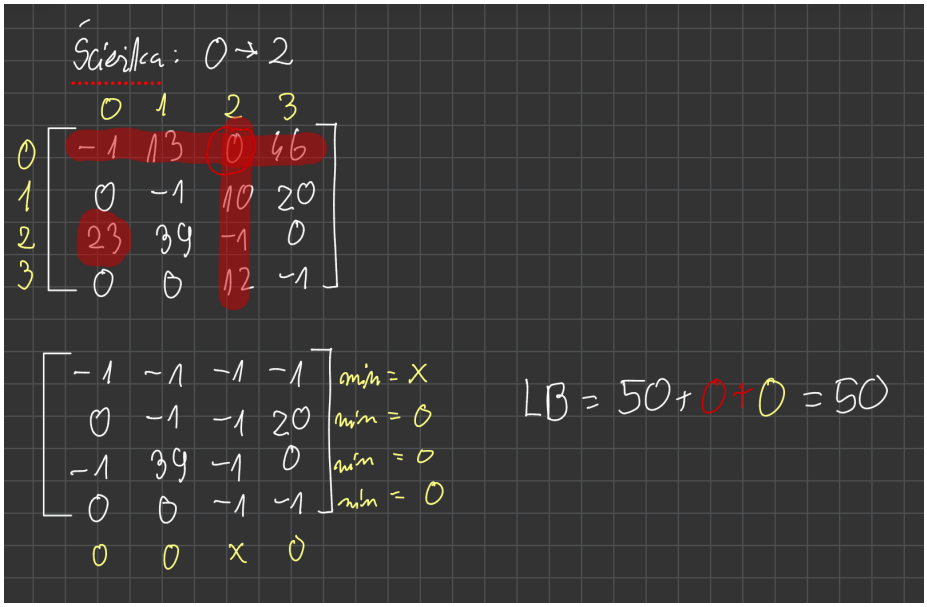

Figure 4: Przykład wyznaczenia LB dla węzła $[0, 2]$

W ten sposób dla przejść z wierzchołka startowego 0 otrzymano następujące wartości:

$$[0,1]\colon LB = 50 + 13 + 10 = 73, \qquad [0,2]\colon LB = 50 + 0 + 0 = 50, \qquad [0,3]\colon LB = 50 + 46 + 33 = 129.$$

Strategia BestFS wybiera w każdym kroku ten węzeł, którego wartość dolnego ograniczenia jest najmniejsza, dlatego w pierwszej kolejności rozwinięto węzeł [0,2]. Po jego rozwinięciu otrzymano dwa kolejne węzły: [0,2,1](LB = 50 + 16 + 43 = 109), [0,2,3] (LB = 50 + 0 + 0 = 50).

Ponownie wybrano gałąź o najmniejszym ograniczeniu [0,2,3]. W następnym kroku zostało już tylko jedno nieodwiedzone miasto i powstała finalna ścieżka [0,2,3,1,0].

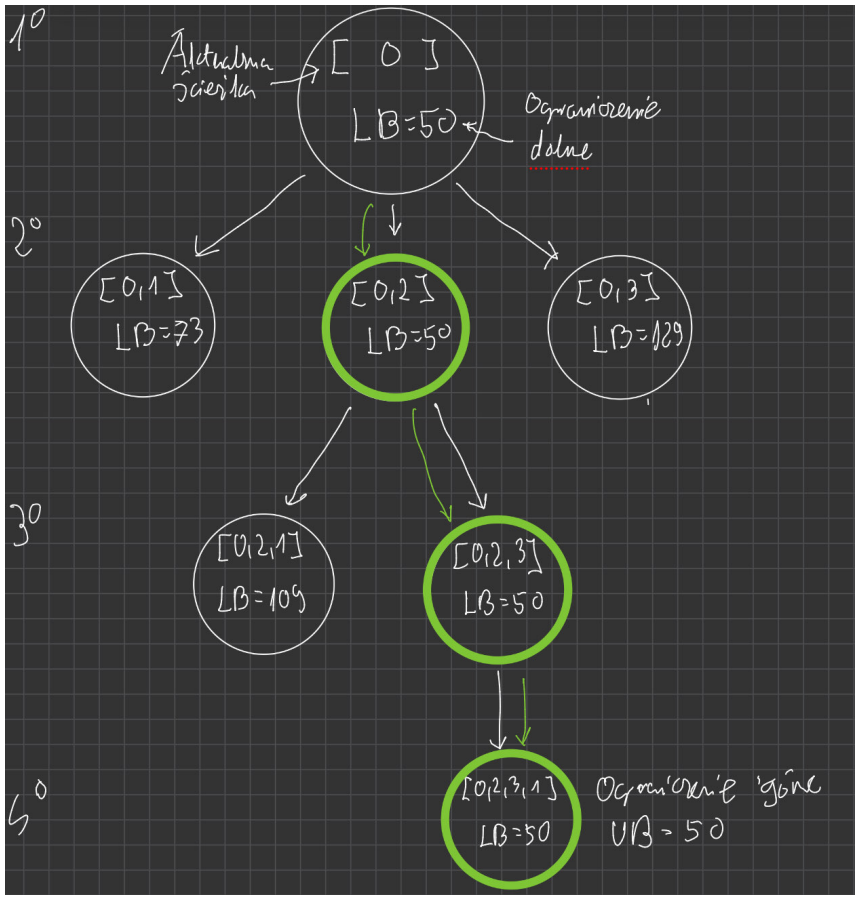

Figure 5: Kolejność rozwijania węzłów przez strategię BestFS

Krok 3. Odcinanie gałęzi i wyznaczenie rozwiązania

Dla uzyskanej trasy [0,2,3,1,0] koszt wynosi 10+12+18+10=50. Wartość ta zostaje nowym ograniczeniem górnym UB=50. Następnie porównano wartość ograniczenia górnego z dolnymi ograniczeniami pozostałych aktywnych gałęzi:

$$[0,1]\colon 73 > 50, \qquad [0,3]\colon 129 > 50, \qquad [0,2,1]\colon 109 > 50.$$

Ponieważ dla wszystkich tych gałęzi zachodzi:

$$LB \geq UB,$$

żadna z nich nie może prowadzić do rozwiązania lepszego od już znalezionego. Wszystkie zostały więc odcięte.

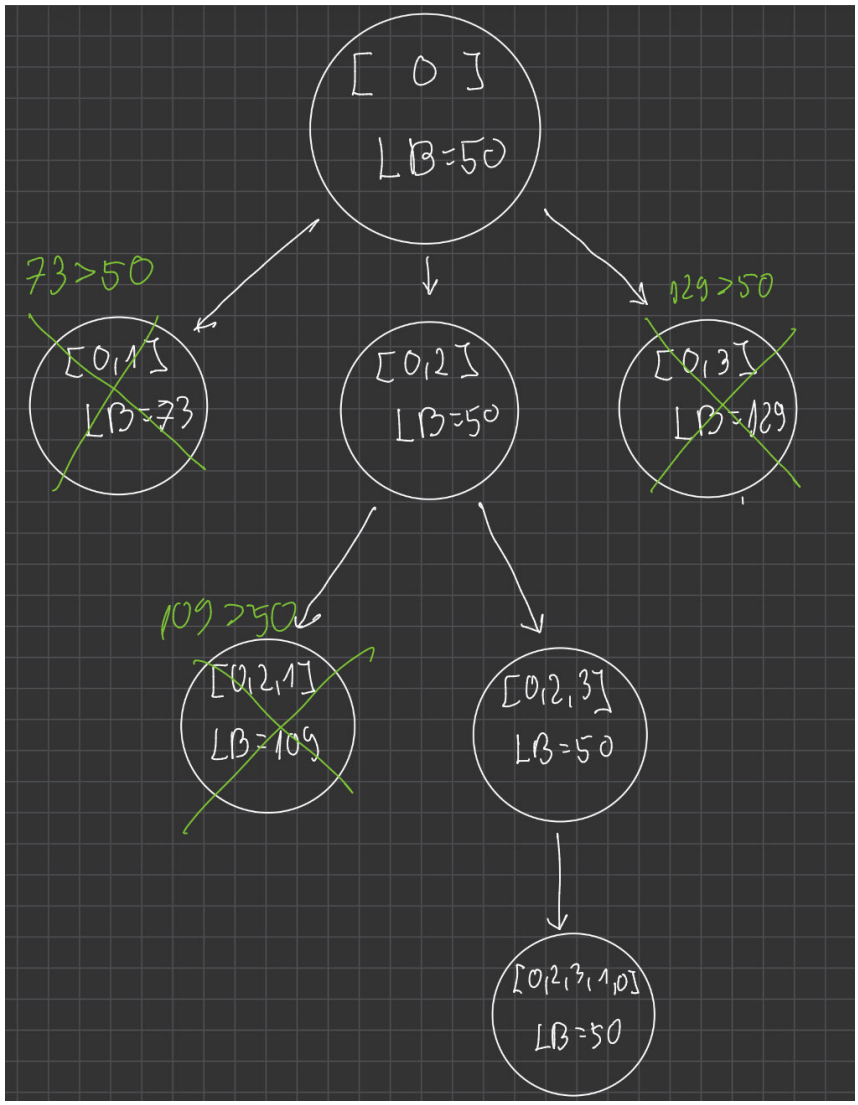

Figure 6: Odcinanie gałęzi po wyznaczeniu ograniczenia górnego $UB = 50$

Ostatecznym znalezionym rozwiązaniem zostaje koszt 50 dla trasy [0,2,3,1,0]

4 Plan eksperymentu

4.1 Cel eksperymentu

Celem eksperymentu była analiza efektywności algorytmu Branch and Bound zastosowanego do asymetrycznego problemu komiwojażera oraz porównanie dwóch strategii przeszukiwania przestrzeni stanów: Breadth-First Search oraz Best First Search. Dodatkowym celem było zbadanie wpływu zastosowania rozwiązania początkowego na czas działania algorytmu. W ramach projektu porównano cztery warianty algorytmu B&B z przeszukiwaniem Breadth First Search oraz Best First Search oba w wariantach bez rozwiązania początkowego oraz z rozwiązaniem początkowym wyznaczonym przez NN. Instancja problemu była reprezentowana w postaci macierzy o rozmairze N x N. Wartość macierzy w każdym punkcie [i][j] oznaczała koszt przejścia z wierzchołka i do wierzchołka j. Wszystkie koszty były wartościami całkowitymi, w testach generowano dla każdej macierzy wartośći zakresu od 10 do 99. Na przekątnej macierzy (koszt przejścia z tego samego wierzchołka) umieszczono wartości -1. Program

umożliwia zarówno wczytanie danych macierzy z pliku jak i generowanie losowych danych dla podanej wielkości macierzy oraz wartości maksymalnej i minimalnej.

4.2 Zakres badanych rozmiarów

Eksperyment podzielono na dwie części. W pierwszej części wykonano porównanie wszystkich czterech wariantów algorytmu dla wspólnego zakresu rozmiarów:

$$N \in \{5, 6, 7, 8, 9, 10, 11, 12\}.$$

Zakres ten dobrano tak, aby możliwe było bezpośrednie porównanie wszystkich metod wspólnie. W drugiej części wykonano badania rozszerzone dla większych instancji:

$$N \in \{13, 14\},$$

jednak tylko dla tych wariantów, które pozostawały wykonalne w rozsądnym czasie, czyli dla:

- BFS z rozwiązaniem początkowym NN,
- BestFS bez rozwiązania początkowego,
- BestFS z rozwiązaniem początkowym NN.

Wariant BFS bez rozwiązania początkowego badano tylko do $N = 12$, ponieważ dla większych instancji osiągał limit czasu i nie zwracał wyników. Następnie, ze względu na bardzo dobrą efektywność strategii BestFS, wykonano dodatkowe pomiary rozszerzone dla tej metody od rozmiaru 15 do $N = 25$.

4.3 Liczba powtórzeń i limit czasu

Dla każdej analizowanej wartości N wykonywano po 100 losowych instancji problemu. Dla każdej instancji uruchamiano wszystkie porównywane warianty algorytmu na tej samej macierzy. Dla pojedynczego uruchomienia algorytmu przyjęto limit czasu wynoszący 5 minut. Jeżeli algorytm nie zakończył pracy w tym czasie dana próba została przerywana. Dodatkowo dla najbardziej kosztownego wariantu, czyli BFS bez rozwiązania początkowego, zastosowano mechanizm wcześniejszego przerwania dalszych prób dla danego rozmiaru po serii kolejnych 10 przerwań. Zastosowane ograniczenie służyło ograniczeniu czasu wykonania pomiarów dla metody która za każdym razem nie mieściła się w limicie czasu.

4.4 Wykonane pomiary

W eksperymencie wykonano następujące pomiary:

- czas wykonania algorytmu dla pojedynczej instancji,
- liczbę przypadków zakończonych poprawnym znalezieniem rozwiązania,
- liczbę przypadków przerwanych po przekroczeniu limitu czasu,
- procent przypadków przerwanych dla danej wartości N,
- średni czas wykonania obliczony na podstawie instancji zakończonych bez przerwania.

Czas mierzono z użyciem `std::chrono::high_resolution_clock`, a wyniki zapisywano w mikrosekundach. Następnie na potrzeby tabel i wykresów wyznaczano średnie wartości dla każdego rozmiaru problemu i każdego wariantu algorytmu.

4.5 Organizacja pomiarów

W eksperymencie mierzono czas wykonania algorytmu dla pojedynczej instancji, liczbę przypadków zakończonych poprawnym znalezieniem rozwiązania, liczbę oraz procent przypadków przerwanych po przekroczeniu limitu czasu, a także średni czas wykonania obliczony na podstawie instancji zakończonych bez przerwania. Czas mierzono z użyciem `std::chrono::high_resolution_clock`, a wyniki zapisywano w mikrosekundach. Dla każdej wartości N generowano losową macierz kosztów, a następnie na tej samej instancji uruchamiano kolejno wszystkie analizowane warianty algorytmu. Z uzyskanych wyników należało określić, która strategia przeszukiwania BreadthFirstSearch czy BestFirstSerach działa szybciej, jak duży wpływ na czas działania ma zastosowanie rozwiązania początkowego, oraz dla których wariantów i wartości N pojawiają się przerwania po przekroczeniu limitu czasu.

5 Wyniki eksperymentów

5.1 Porównanie czasów działania badanych wariantów

W tabeli oraz na wykresie przedstawiono średnie czasy działania wszystkich badanych wariantów algorytmu Branch and Bound. Dla wartości od 5 do 11 możliwe było bezpośrednie porównanie wszystkich czterech wariantów. Dla rozmiaru instancji 12 i większych problem stawał się zbyt czasochłonny dla algorytmu BFS bez rozwiązania wstępnego NN i dla instancji o rozmiarze 12 osiągnął 100% przerwań wykonywania związanych z ograniczeniem działania algorytmów przez maksymalnie 5 minut. Z tego względu dla rozmiarów 13 i 14 pomiary dla BFS bez wstępnego rozwiązania heurystycznego zostały pominięte.

Table 1: Średni czas działania badanych wariantów algorytmu

N	BFS bez NN [ms]	BFS + NN [ms]	BestFS bez NN [ms]	BestFS + NN [ms]
5	0,56259	0,13355	0,12189	0,07961
6	3,4805	0,66547	0,26295	0,20919
7	12,30203	1,14589	0,25902	0,2004
8	46,60104	2,92525	0,23721	0,18812
9	412,85499	9,65969	0,45983	0,35464
10	4132,96937	47,63674	0,71269	0,60408
11	52640,06601	182,39387	1,31981	1,12481
12	–	694,93146	2,57155	2,17769
13	–	10261,1311	4,33876	3,36013
14	–	17496,69178	9,02667	7,37949

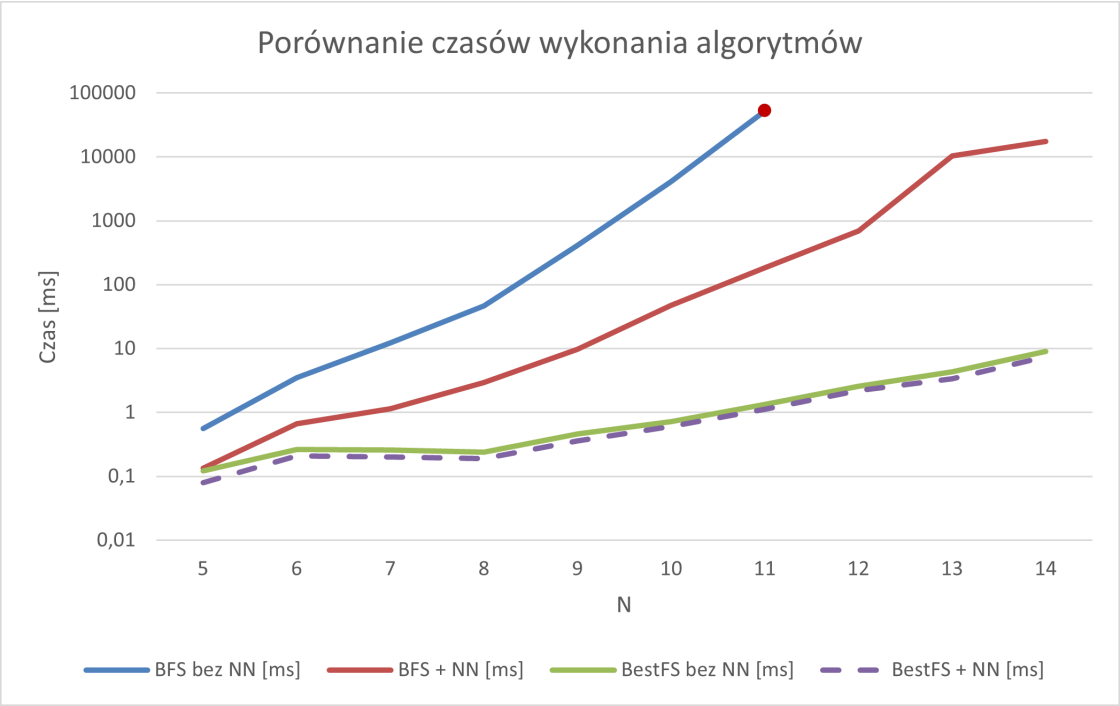

Figure 7: Porównanie czasów wykonania badanych wariantów algorytmu

Na podstawie przedstawionych danych widać, że najwolniejszym wariantem był BFS bez rozwiązania początkowego. Już dla $N = 11$ jego średni czas działania był o kilka rzędów wielkości większy od pozostałych metod. Zastosowanie rozwiązania początkowego wyznaczonego heurystycznie przez algorytm NN znacząco przyspieszało działanie BFS. Jednocześnie strategia BestFS okazała się wyraźnie efektywniejsza od BFS zarówno bez rozwiązania początkowego, jak i z jego wykorzystaniem. Najkrótsze czasy działania uzyskano dla wariantu BestFS ze wstępny rozwiązaniem heurystycznym.

Dodatkowo zmierzono procent przerwań algorytmów po przekroczeniu limitu czasu wynoszącego 5 minut.

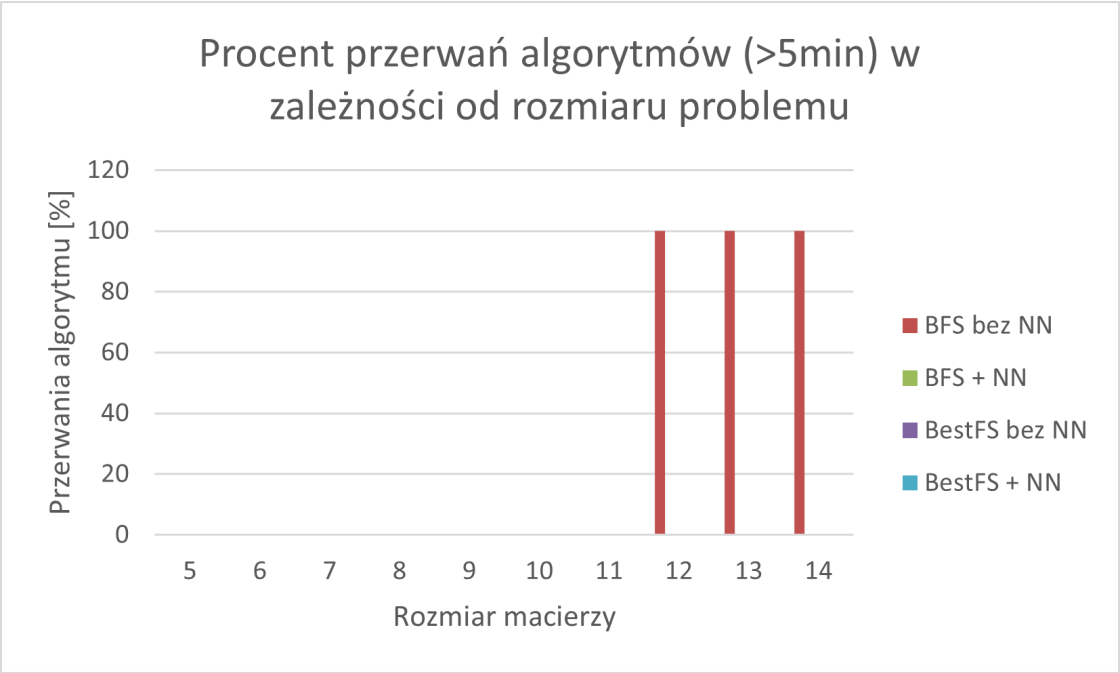

Figure 8: Procent przerwanych wykonań algorytmów w zależności od rozmiaru problemu

Na podstawie wykresu widać, że przerwania wykonywania pojawiły się wyłącznie dla wariantu BFS bez rozwiązania początkowego. Dla $N = 12$ i wyższych rozmiarów odsetek przerwanych prób dla BreadthFirstSearch wyniósł 100

5.2 Wpływ rozwiązania początkowego

W celu zbadania wpływu rozwiązania początkowego porównano osobno wyniki dla strategii BFS oraz BestFS w dwóch wariantach: bez wstępnego rozwiązania oraz z rozwiązaniem początkowym wyznaczonym przy użyciu algorytmu Nearest Neighbour.

Table 2: Wpływ rozwiązania początkowego na czas działania BFS

N	BFS bez NN [ms]	BFS + NN [ms]	Przyspieszenie [x]
5	0,56259	0,13355	4,21
6	3,4805	0,66547	5,23
7	12,30203	1,14589	10,74
8	46,60104	2,92525	15,93
9	412,85499	9,65969	42,74
10	4132,96937	47,63674	86,76
11	52640,06601	182,39387	288,61
12	–	694,93146	–
13	–	10261,1311	–
14	–	17496,69178	–

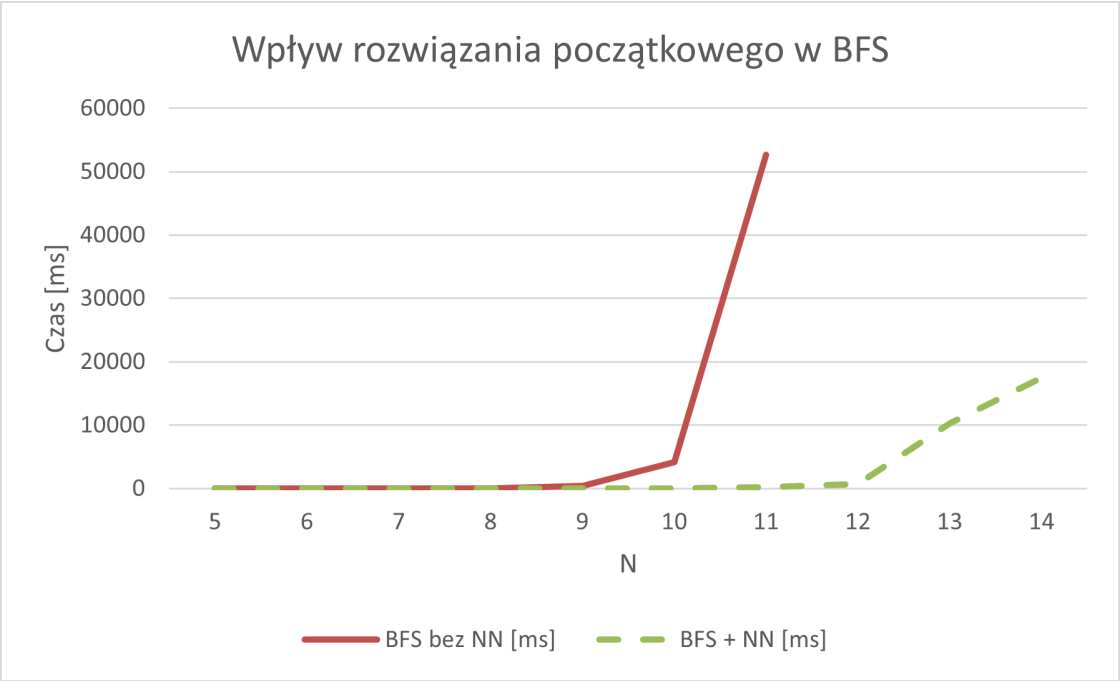

Figure 9: Wpływ rozwiązania początkowego na czas działania strategii BFS

Dla strategii BFS wpływ rozwiązania początkowego był bardzo duży. Wariant BFS + NN był w całym badanym zakresie wyraźnie szybszy od wariantu BFS bez rozwiązania początkowego. Oznacza to, że początkowe ograniczenie górne wyznaczone przez heurystykę NN znacząco zwiększa skuteczność odcinania niepotrzebnych gałęzi. Bez zastosowania wstępnego rozwiązania strategia BFS bardzo szybko staje się nieefektywna wraz ze wzrostem rozmiaru problemu.

Table 3: Wpływ rozwiązania początkowego na czas działania BestFS

N	BestFS bez NN [ms]	BestFS + NN [ms]	Przyspieszenie [x]
5	0,12189	0,07961	1,53
6	0,26295	0,20919	1,26
7	0,25902	0,2004	1,29
8	0,23721	0,18812	1,26
9	0,45983	0,35464	1,30
10	0,71269	0,60408	1,18
11	1,31981	1,12481	1,17
12	2,57155	2,17769	1,18
13	4,33876	3,36013	1,29
14	9,02667	7,37949	1,22

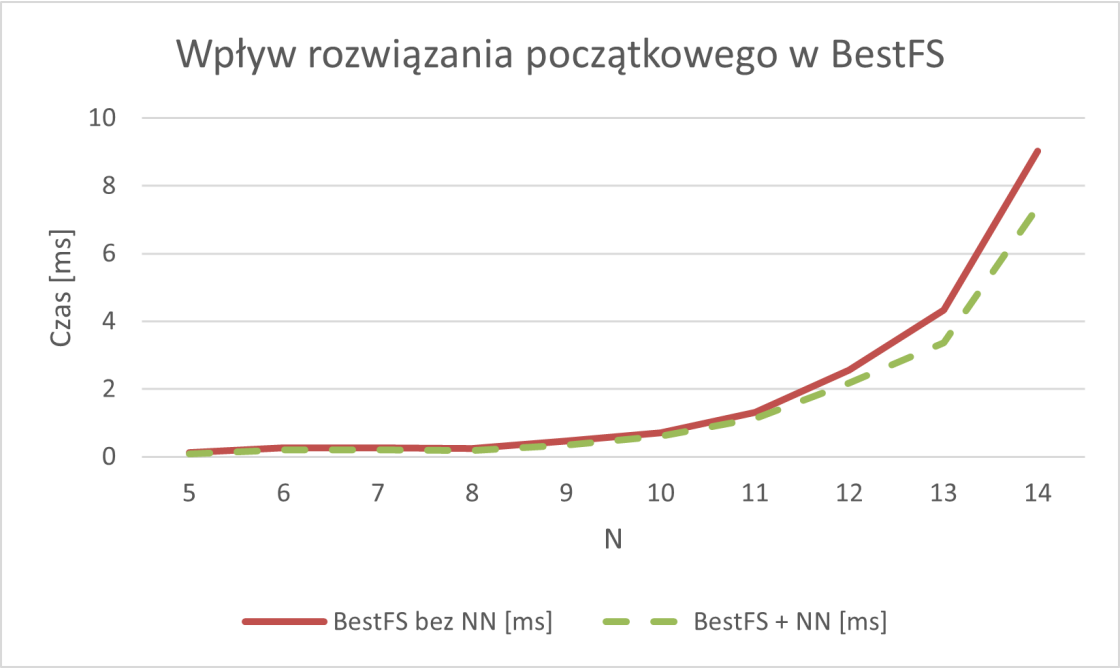

Figure 10: Wpływ rozwiązania początkowego na czas działania strategii BestFS

W przypadku strategii BestFS również zaobserwowano pozytywny wpływ wykorzystania rozwiązania początkowego, jednak różnica między wariantem bez NN i z NN była znacznie mniejsza niż w przypadku BFS. Oznacza to, że sama strategia wyboru węzła o najmniejszym dolnym ograniczeniu jest już bardzo skuteczna i pozwala szybciej kierować przeszukiwanie do obszarów drzewa o największym potencjale.

Na podstawie eksperymentu można stwierdzić, że zastosowanie rozwiązania początkowego poprawia efektywność obu strategii, jednak jego znaczenie jest zdecydowanie większe dla BFS niż dla BestFS.

5.3 Strategia B&B BestFirstSearch dla większych wartości N

Ze względu na bardzo szybki wzrost czasu działania strategii BFS dla większych instancji problemu wykonano dodatkowe pomiary dla większych instancji problemu przy użyciu wariantu BestFirstSearch.

Table 4: Średni czas działania strategii BestFS

N	BestFS bez NN [ms]	BestFS + NN [ms]
15	13,60	12,31
16	20,27	18,25
17	28,32	26,32
18	51,51	49,95
19	57,65	55,55
20	142,68	136,61
21	192,33	183,68
22	327,07	296,59
23	629,20	604,76
24	715,09	673,68
25	1212,68	1195,04

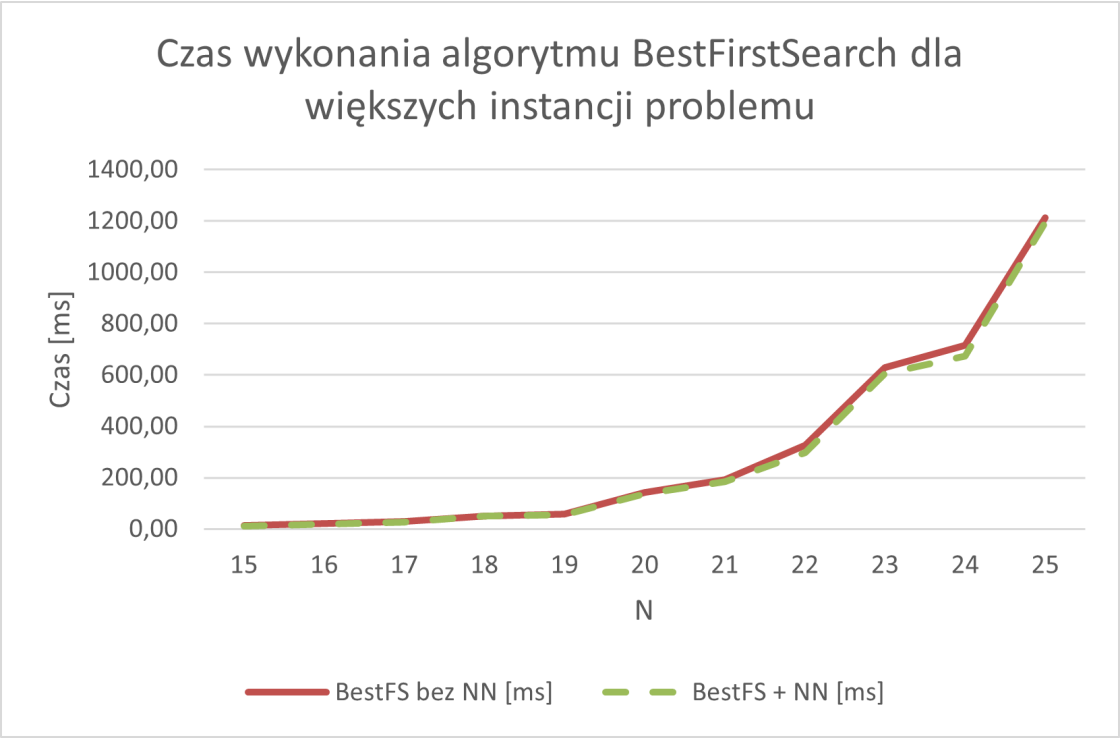

Figure 11: Czas wykonania algorytmu BestFirstSearch dla większych instancji problemu

Strategia B&B BestFS zachowuje dobrą efektywność również dla większych rozmiarów problemu. Wraz ze wzrostem N czas działania stopniowo rośnie, jednak nawet dla $N = 25$ średni czas wykonania pozostaje na poziomie około 1,2 sekundy. Oznacza to, że w praktyce strategia BestFS nadal pozostaje użyteczna dla znacznie większych instancji niż te, dla których możliwe było skuteczne zastosowanie BFS. Uzyskane wyniki pokazują wyraźną przewagę strategii BestFS nad algorytmem pełnego przeglądu. W poprzednim projekcie algorytm Brute Force już dla znacznie mniejszych wartości N osiągał czasy rzędu setek sekund (dla N=14 230 sekund). Przyczyną tak dużej różnicy jest fakt, że algorytm nie analizuje wszystkich możliwych permutacji odwiedzania miast, lecz odrzuca te gałęzie drzewa rozwiązań, które już na podstawie oszacowanego dolnego ograniczenia nie mogą prowadzić do rozwiązania lepszego od aktualnie najlepszego. Dodatkowo strategia BestFS rozwija w pierwszej kolejności te węzły, które mają najmniejszą wartość ograniczenia dolnego, dzięki czemu szybciej dociera do najbardziej obiecujących obszarów przestrzeni rozwiązań. W efekcie liczba rzeczywiście analizowanych przypadków jest nieporównywalnie mniejsza niż w algorytmie Brute Force, co bezpośrednio przekłada się na dużo krótszy czas działania.

Na podstawie uzyskanych wyników oraz dodatkowych prób dla większych instancji można oszacować, że BestFS osiąga limit czasu 5 minut w okolicach rozmiaru instancji 36-37. Dodatkowe pojedyncze testy pokazały, że dla tego samego rozmiaru problemu czas działania może silnie zależeć od konkretnej instancji – przykładowo dla $N = 36$ uzyskano czas około 46 sekund, natomiast dla $N = 37$ w jednej próbie czas wyniósł około 7 sekund, a w innej algorytm nie zakończył działania nawet po czasie wyraźnie dłuższym niż przyjęty limit 5 minut i powodował bardzo duże zużycie pamięci operacyjnej. ..

5.4 Podsumowanie

Z wyników eksperymentu wynika, że w alorytmie B&B strategia przeszukiwania oraz zastosowanie rozwiązania początkowego mają istotny wpływ na czas działania algorytmu. Spośród wszystkich badanych wariantów najlepsze wyniki uzyskała strategia BestFS w połączeniu z rozwiązaniem początkowym wyznaczonym przez algorytm Nearest Neighbour.

Przeprowadzone eksperymenty pokazały, że strategia BFS bez rozwiązania początkowego bardzo szybko staje się niepraktyczna wraz ze wzrostem rozmiaru problemu. Już od $N = 12$ odsetek przerwanych uruchomień dla tego wariantu wynosił 100%, a sam algorytm w takiej konfiguracji wypadał gorzej od algorytmu Brute Force. Strategia przeszukiwania BFS nie jest dobrym wyborem dla metody Branch and Bound, ponieważ zbyt długo rozwija dużą liczbę mało obiecujących węzłów, co znacząco ogranicza skuteczność odcinania gałęzi i prowadzi do bardzo szybkiego wzrostu czasu działania. Zastosowanie rozwiązania początkowego przy pomocy Nearest Neigbour znacząco przyśpieszyła działanie algorytmu, jednak dalej pozostał on wysoce nieefektywny w porównaniu z BestFS.

W przypadku strategii BestFS wpływ rozwiązania początkowego również był pozytywny, lecz znacznie mniejszy niż dla BFS. Sama metoda wyboru węzła o najmniejszym dolnym ograniczeniu jest już sama w sobie bardzo skuteczna.

W porównaniu z algorytmem Brute Force z poprzedniego projektu uzyskano bardzo dużą poprawę wydajności. Podczas gdy Brute Force dla $N = 14$ osiągał czas rzędu 230 sekund, strategia BestFS dla większych instancji nadal działała w czasie liczonym w milisekundach lub pojedynczych sekundach. Zastosowanie metody podziału i ograniczeń oraz odpowiedniej strategii wyboru węzłów znacząco zwiększa zakres instancji, które można rozwiązać w akceptowalnym czasie.

6 Literatura

[1] Travelling salesman problem, Wikipedia: `https://en.wikipedia.org/wiki/Travelling_salesman_problem`, dostęp: 23.03.2026.
[2] Algorytmy decyzyjne i teoria złożoności, plik `komiw_little.pdf`, udostępnionY przez prowadzącego, dostęp: 21.04.2026.